

Selection Sort

Pseudocode

```

SELECTION_SORT(A, n)
  for i = 0 to n-1
    min = i
    for j = i+1 to n-1
      if A[j] < A[min]
        min = j
    swap A[i], A[min]

```

Coding Phyton

```

def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

data = [64, 25, 12, 22, 11]
print(selection_sort(data))

[11, 12, 22, 25, 64]

```

Analisa Kompleksitas :

Best Case : $O(n^2)$

Average Case : $O(n^2)$

Worst Case : $O(n^2)$

Insertion Sort

Pseudocode

```

INSERTION-SORT(A)
1. for i ← 1 to panjang(A)-1
2.   key ← A[i]
3.   j ← i-1
4.   while j ≥ 0 dan A[j] > key
5.     A[j+1] ← A[j]
6.     j ← j-1
7.   A[j+1] ← key

```

Coding Phyton

```

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr

data = [19, 25, 13, 3, 7]
print(insertion_sort(data))

[3, 7, 13, 19, 25]

```

Analisa Kompleksitas :

Best Case : $O(n)$

Average Case : $O(n^2)$

Worst Case : $O(n^2)$

Merge Sort

Pseudocode

```

MERGE-SORT(A, left, right)
    if left < right
        mid ← (left + right) / 2
        MERGE-SORT(A, left, mid)
        MERGE-SORT(A, mid+1, right)
        MERGE(A, left, mid, right)

MERGE(A, left, mid, right)

```

Coding Phyton

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result += left[i:]
    result += right[j:]

    return result

data = [38, 27, 43, 3, 9, 82, 10]

print("Sebelum sorting:", data)
print("Sesudah sorting :", merge_sort(data))

```

```

Sebelum sorting: [38, 27, 43, 3, 9, 82, 10]
Sesudah sorting : [3, 9, 10, 27, 38, 43, 82]

```

Analisa Kompleksitas :

Best Case : $O(n \log n)$

Average Case : $O(n \log n)$

Worst Case : $O(n \log n)$

Quick Sort

Pseudocode

```

QUICK_SORT(A, lo, hi)
    if lo >= hi
        return
    pivot = PARTITION(A, lo, hi)
    QUICK_SORT(A, lo, pivot-1)
    QUICK_SORT(A, pivot+1, hi)

PARTITION(A, lo, hi) // 3-way partitioning
    lt = lo, i = lo+1, gt = hi

```

```
v = A[lo]
while i <= gt
    if A[i] < v
        swap A[lt], A[i]; lt++; i++
    else if A[i] > v
        swap A[i], A[gt]; gt--
    else
        i++
swap A[lo], A[lt]
return lt
```

Coding Phyton

```
def quick_sort(arr):

    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

data = [100, 77, 85, 93, 10, 50]
print(quick_sort(data))
```

```
[10, 50, 77, 85, 93, 100]
```

Analisa Kompleksitas :

Best Case : $O(n \log n)$

Average Case : $O(n \log n)$

Worst Case : $O(n^2)$